

Classic Synchronization Problems

This note covers three famous operating-system synchronization problems:

- Producer-Consumer Problem
- Readers-Writers Problem
- Dining Philosophers Problem

These problems are extremely common in OS interviews because they test whether you understand:

- Processes and threads
 - Race conditions
 - Critical sections
 - Mutual exclusion
 - Semaphores
 - Mutex locks
 - Condition variables
 - Deadlock
 - Starvation
 - Bounded waiting
 - Synchronization design
-

1. Producer-Consumer Problem

1.1 Basic Idea

The Producer-Consumer Problem describes a situation where:

- One or more producer processes generate data.
- One or more consumer processes use that data.
- Both producers and consumers share a common buffer.

The producer places items into the buffer. The consumer removes items from the buffer.

The main challenge is to make sure producers and consumers access the shared buffer safely.

This problem is also called the Bounded Buffer Problem when the buffer has a fixed size.

1.2 Real-Life Examples

Examples of producer-consumer systems:

- Keyboard input buffer:
 - Keyboard produces characters.
 - An application consumes characters.
- Print spooler:
 - Applications produce print jobs.

- Printer consumes print jobs.
- Web server:
 - Network accepts incoming requests.
 - Worker threads process requests.
- Video streaming:
 - Downloader thread produces video chunks.
 - Player thread consumes video chunks.
- Operating-system pipes:
 - One process writes data.
 - Another process reads data.

1.3 The Shared Buffer

A buffer is a temporary storage area.

For example, suppose the buffer size is 5:

```
Buffer: [ _ , _ , _ , _ , _ ]
```

If the producer inserts three items:

```
Buffer: [ A , B , C , _ , _ ]
```

If the consumer removes one item:

```
Buffer: [ _ , B , C , _ , _ ]
```

In actual implementation, a circular queue is commonly used.

1.4 Problems Without Synchronization

If producers and consumers access the buffer without synchronization, several issues can occur.

Race Condition

A race condition occurs when multiple threads access shared data at the same time and the final result depends on timing.

Example:

- Producer checks that buffer is not full.
- Consumer removes an item.
- Another producer inserts an item.
- First producer now inserts based on stale information.

This can corrupt the buffer.

Buffer Overflow

If the producer keeps producing when the buffer is full, it may overwrite existing data.

Example:

```
Buffer size = 3  
Buffer: [ A , B , C ]
```

If producer adds D without waiting:

```
Data may be overwritten or lost.
```

Buffer Underflow

If the consumer tries to consume when the buffer is empty, it may read invalid data.

Example:

```
Buffer: [ _ , _ , _ ]
```

Consumer tries to remove an item, but no item exists.

1.5 Requirements of the Solution

A correct solution must ensure:

1. Mutual exclusion: Only one process should access the shared buffer at a time.
2. No overflow: A producer must wait if the buffer is full.
3. No underflow: A consumer must wait if the buffer is empty.
4. Progress: If work can be done, some process should be able to proceed.
5. Bounded waiting: No process should wait forever if it is eligible to proceed.

1.6 Semaphore-Based Solution

The classic solution uses three synchronization variables:

```
semaphore mutex = 1;  
semaphore empty = n;
```

```
semaphore full = 0;
```

Where:

- `mutex` protects the buffer.
- `empty` counts the number of empty buffer slots.
- `full` counts the number of filled buffer slots.
- `n` is the size of the buffer.

1.7 Meaning of Each Semaphore

mutex

The `mutex` semaphore provides mutual exclusion.

Only one producer or consumer can enter the critical section at a time.

Initial value:

```
mutex = 1;
```

Because initially, the buffer is free to access.

empty

The `empty` semaphore counts empty slots in the buffer.

Initial value:

```
empty = n;
```

Because initially, all buffer slots are empty.

If the buffer size is 5:

```
empty = 5;
```

full

The `full` semaphore counts filled slots in the buffer.

Initial value:

```
full = 0;
```

Because initially, no item exists in the buffer.

1.8 Producer Algorithm

```
do {
    item = produce_item();

    wait(empty);
    wait(mutex);

    insert_item(item);

    signal(mutex);
    signal(full);

} while (true);
```

1.9 Consumer Algorithm

```
do {
    wait(full);
    wait(mutex);

    item = remove_item();

    signal(mutex);
    signal(empty);

    consume_item(item);

} while (true);
```

1.10 Step-by-Step Explanation

Assume buffer size is 3.

Initial values:

```
empty = 3
full = 0
mutex = 1
```

Producer Produces First Item

Producer calls:

```
wait(empty);
```

Now:

```
empty = 2
```

Producer calls:

```
wait(mutex);
```

Now:

```
mutex = 0
```

Producer inserts item into buffer.

Then:

```
signal(mutex);  
signal(full);
```

Now:

```
mutex = 1  
full = 1
```

Consumer Consumes Item

Consumer calls:

```
wait(full);
```

Now:

```
full = 0
```

Consumer calls:

```
wait(mutex);
```

Now:

```
mutex = 0
```

Consumer removes item.

Then:

```
signal(mutex);  
signal(empty);
```

Now:

```
mutex = 1  
empty = 3
```

1.11 Why the Order Matters

The producer must do:

```
wait(empty);  
wait(mutex);
```

It should not do:

```
wait(mutex);  
wait(empty);
```

Why?

Suppose the buffer is full.

If producer first locks `mutex` and then waits on `empty`, the producer sleeps while holding the buffer lock.

Then the consumer cannot acquire `mutex` to remove an item.

Result:

Deadlock

So the producer must first check whether an empty slot exists, then lock the buffer.

Similarly, the consumer must do:

```
wait(full);  
wait(mutex);
```

It should not lock the buffer before checking whether an item exists.

1.12 Important Interview Points

Interviewers may ask:

Why do we need three semaphores?

Because each solves a different problem:

- `mutex` protects the critical section.
- `empty` prevents buffer overflow.
- `full` prevents buffer underflow.

Can the problem be solved with only mutex?

Not correctly for a bounded buffer.

A mutex can protect the buffer, but it cannot by itself tell a producer to wait when the buffer is full or tell a consumer to wait when the buffer is empty.

What happens if `signal(full)` is forgotten?

Consumers may wait forever because they are never told that an item is available.

What happens if `signal(empty)` is forgotten?

Producers may wait forever because they are never told that a slot became empty.

Is this problem about mutual exclusion only?

No.

It involves both:

- Mutual exclusion
- Condition synchronization

Condition synchronization means a process waits until a condition becomes true.

Examples:

- Producer waits until buffer is not full.
- Consumer waits until buffer is not empty.

1.13 Common Mistakes

Common errors in producer-consumer solutions:

- Using only one semaphore.
- Forgetting to protect the buffer with a mutex.
- Calling `wait(mutex)` before `wait(empty)` in the producer.
- Calling `wait(mutex)` before `wait(full)` in the consumer.
- Forgetting to signal after modifying the buffer.
- Using busy waiting unnecessarily.
- Not handling multiple producers and consumers.

1.14 Summary

The Producer-Consumer Problem teaches:

- How to synchronize shared buffer access.
- How to avoid buffer overflow.
- How to avoid buffer underflow.
- How semaphores can represent available resources.
- Why locking order matters.

2. Readers-Writers Problem

2.1 Basic Idea

The Readers-Writers Problem deals with a shared data object, such as:

- A file
- A database record
- A memory data structure
- A shared document

There are two types of processes:

- Readers:
 - Only read the shared data.
 - Do not modify it.
- Writers:
 - Modify the shared data.
 - Require exclusive access.

The main idea:

- Multiple readers can read at the same time.
- Only one writer can write at a time.
- If a writer is writing, no reader can read.
- If a reader is reading, a writer should not write.

2.2 Real-Life Examples

Examples:

- Online library system:
 - Many users can view book details.
 - Only admin can update book details.
- Bank account record:
 - Many systems may read balance.
 - Only one transaction should update balance.
- Configuration file:
 - Many programs may read settings.
 - Only one update should happen at a time.
- Database:
 - Many queries can read.
 - Writes require exclusive access.

2.3 Why Readers Can Share Access

Readers only observe data.

If two readers read at the same time, they do not corrupt the data.

Example:

```
Reader 1 reads account balance = 5000  
Reader 2 reads account balance = 5000
```

No problem occurs.

But if a writer updates the balance at the same time:

```
Writer changes balance from 5000 to 7000  
Reader reads halfway through update
```

The reader may see inconsistent data.

2.4 Requirements of the Problem

A correct solution should ensure:

1. Multiple readers may access the shared data together.
2. Only one writer may access the shared data at a time.
3. A writer must have exclusive access.
4. No reader should read while a writer is writing.
5. No writer should write while readers are reading.
6. Starvation should be avoided if fairness is required.

2.5 Variations of Readers-Writers Problem

There are three major versions.

2.6 First Readers-Writers Problem

The first version gives priority to readers.

Rule:

No reader should wait unless a writer is already writing.

Meaning:

- If readers are already reading, new readers may enter.
- A waiting writer must wait until all readers leave.

Problem:

- Writers can starve.

If readers keep arriving continuously, the writer may never get a chance.

2.7 Second Readers-Writers Problem

The second version gives priority to writers.

Rule:

Once a writer is waiting, no new reader should start reading.

Meaning:

- If a writer is waiting, new readers are blocked.
- Existing readers finish.

- Then writer writes.

Problem:

- Readers can starve if writers keep arriving continuously.

2.8 Third Readers-Writers Problem

The third version aims for fairness.

Rule:

Neither readers nor writers should starve.

This is often implemented with a queue or fair lock.

2.9 Reader-Priority Solution Using Semaphores

Variables:

```
semaphore mutex = 1;
semaphore wrt = 1;
int read_count = 0;
```

Where:

- `mutex` protects `read_count`.
- `wrt` controls access to the shared data.
- `read_count` stores the number of active readers.

2.10 Reader Algorithm

```
do {
    wait(mutex);
    read_count++;

    if (read_count == 1) {
        wait(wrt);
    }

    signal(mutex);

    read_data();

    wait(mutex);
    read_count--;

    if (read_count == 0) {
```

```
        signal(wrt);  
    }  
  
    signal(mutex);  
  
} while (true);
```

2.11 Writer Algorithm

```
do {  
    wait(wrt);  
  
    write_data();  
  
    signal(wrt);  
  
} while (true);
```

2.12 Explanation of Reader-Priority Solution

The first reader locks the shared resource for readers.

```
if (read_count == 1) {  
    wait(wrt);  
}
```

This prevents writers from entering while readers are active.

Other readers do not wait on `wrt`. They only update `read_count`.

The last reader releases the shared resource.

```
if (read_count == 0) {  
    signal(wrt);  
}
```

This allows a waiting writer to enter.

2.13 Why We Need `mutex`

The variable `read_count` is shared among readers.

Without `mutex`, two readers may update it incorrectly.

Example:

```
read_count = 0
Reader 1 reads read_count = 0
Reader 2 reads read_count = 0
Reader 1 increments to 1
Reader 2 increments to 1
```

Actual number of readers is 2, but `read_count` becomes 1.

This is a race condition.

So `mutex` is needed to protect updates to `read_count`.

2.14 Why the First Reader Locks `wrt`

The first reader entering means the shared data is now being read.

So writers must be blocked.

If every reader did `wait(wrt)`, then only one reader could read at a time, which defeats the purpose.

Therefore:

- First reader locks `wrt`.
- Other readers share the read access.
- Last reader unlocks `wrt`.

2.15 Why the Last Reader Releases `wrt`

As long as at least one reader is active, a writer cannot write.

When `read_count` becomes 0, all readers have left.

Then the writer can safely enter.

So the last reader performs:

```
signal(wrt);
```

2.16 Starvation in Reader-Priority Solution

Writer starvation can happen.

Example:

1. Reader R1 starts reading.
2. Writer W1 arrives and waits.
3. Reader R2 arrives.
4. R2 is allowed to read because no writer is currently writing.
5. Reader R3 arrives.

6. R3 is also allowed.
7. This continues forever.
8. W1 never writes.

This is starvation.

2.17 Writer-Priority Concept

In a writer-priority solution, once a writer is waiting, new readers are blocked.

This prevents writer starvation.

General idea:

```
If writer is waiting:  
    block new readers  
Allow current readers to finish  
Allow writer to write
```

This requires additional semaphores or locks.

2.18 Fair Readers-Writers Concept

A fair solution serves readers and writers roughly in arrival order.

This usually uses:

- A queue
- A turnstile semaphore
- A fair read-write lock

The idea is:

```
Processes line up.  
Readers may batch together when safe.  
Writers get exclusive turns.
```

2.19 Read-Write Lock

Modern systems often provide a read-write lock.

A read-write lock has two modes:

- Read lock:
 - Multiple threads can hold it simultaneously.
- Write lock:

- Only one thread can hold it.
- No readers can hold it at the same time.

Examples:

- POSIX: `pthread_rwlock_t`
- Java: `ReentrantReadWriteLock`
- C++: `std::shared_mutex`

2.20 Important Interview Points

Why can multiple readers enter together?

Because reading does not modify shared data.

Why does a writer need exclusive access?

Because writing changes data and can create inconsistent state if mixed with reads or writes.

What is the purpose of `read_count`?

It tracks how many readers are currently reading.

Why protect `read_count` with `mutex`?

Because multiple readers update it concurrently.

What is writer starvation?

It happens when writers wait indefinitely because readers continuously enter.

Which variation is best?

It depends on the system:

- Reader-priority is good when reads are frequent and writes are rare.
- Writer-priority is good when writes must not be delayed too long.
- Fair solution is best when starvation must be avoided.

2.21 Common Mistakes

Common errors:

- Allowing writers and readers together.
- Forgetting to protect `read_count`.
- Making every reader wait on `wrt`, which prevents concurrent reading.
- Ignoring starvation.
- Confusing mutual exclusion with reader sharing.
- Assuming reader-priority is always best.

2.22 Summary

The Readers-Writers Problem teaches:

- Shared reading can be safe.
- Writing requires exclusive access.
- Reader priority can starve writers.
- Writer priority can starve readers.
- Fairness may require additional mechanisms.

3. Dining Philosophers Problem

3.1 Basic Idea

The Dining Philosophers Problem was introduced by Edsger Dijkstra.

It models a group of philosophers sitting around a circular table.

Each philosopher alternates between:

- Thinking
- Eating

There is one chopstick between each pair of philosophers.

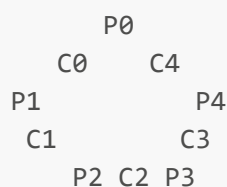
To eat, a philosopher needs two chopsticks:

- Left chopstick
- Right chopstick

The challenge is to design a synchronization solution so that philosophers can eat without causing deadlock or starvation.

3.2 Table Arrangement

For 5 philosophers:



Another simple view:

```
P0 sits between C4 and C0
P1 sits between C0 and C1
P2 sits between C1 and C2
```

```
P3 sits between C2 and C3
P4 sits between C3 and C4
```

Each chopstick is shared by two neighboring philosophers.

3.3 What This Problem Represents

This problem represents resource allocation among competing processes.

Philosophers represent processes. Chopsticks represent resources.

Each process needs multiple resources to proceed.

This appears in real systems such as:

- Multiple processes needing multiple files.
- Transactions needing multiple database locks.
- Threads needing multiple mutexes.
- Programs needing CPU, memory, and I/O resources.

3.4 The Main Challenge

The main challenge is:

```
How do we let philosophers eat while avoiding deadlock and starvation?
```

3.5 Naive Solution

A simple but incorrect solution:

```
do {
    think();

    wait(left_chopstick);
    wait(right_chopstick);

    eat();

    signal(left_chopstick);
    signal(right_chopstick);
} while (true);
```

3.6 Why the Naive Solution Can Deadlock

Suppose all philosophers become hungry at the same time.

Each philosopher picks up their left chopstick.

Now:

```
P0 has C4 and waits for C0
P1 has C0 and waits for C1
P2 has C1 and waits for C2
P3 has C2 and waits for C3
P4 has C3 and waits for C4
```

Every philosopher is holding one chopstick and waiting for another.

No one can proceed.

This is deadlock.

3.7 Deadlock Conditions

Dining Philosophers demonstrates the four necessary conditions for deadlock.

1. Mutual Exclusion

Each chopstick can be held by only one philosopher at a time.

2. Hold and Wait

Each philosopher holds one chopstick while waiting for another.

3. No Preemption

A chopstick cannot be forcibly taken away. It must be released voluntarily.

4. Circular Wait

Each philosopher waits for a chopstick held by the next philosopher in a cycle.

If all four conditions hold, deadlock is possible.

3.8 Solution Goals

A good solution should ensure:

1. No two neighboring philosophers eat at the same time.
2. No deadlock occurs.
3. No philosopher starves forever.
4. The solution should allow maximum possible concurrency.

3.9 Solution 1: Allow at Most Four Philosophers to Sit

For 5 philosophers, allow only 4 philosophers to try eating at a time.

Use a semaphore:

```
semaphore room = 4;
```

Philosopher:

```
do {  
    think();  
  
    wait(room);  
  
    wait(left_chopstick);  
    wait(right_chopstick);  
  
    eat();  
  
    signal(left_chopstick);  
    signal(right_chopstick);  
  
    signal(room);  
  
} while (true);
```

3.10 Why This Prevents Deadlock

If only 4 philosophers are allowed to compete for 5 chopsticks, at least one philosopher is not holding a chopstick.

This means at least one chopstick will remain available somewhere in the system.

That breaks the circular waiting condition.

So deadlock is avoided.

3.11 Advantage of Room Solution

Advantages:

- Simple.
- Easy to understand.
- Prevents deadlock.
- Allows multiple philosophers to eat if they are not neighbors.

3.12 Disadvantage of Room Solution

Disadvantages:

- Does not guarantee perfect fairness by itself.

- Limits concurrency slightly.
- A philosopher could still starve depending on semaphore scheduling.

3.13 Solution 2: Pick Chopsticks in an Ordered Manner

Another solution is to impose an ordering on chopsticks.

Rule:

Always pick the lower-numbered chopstick first, then the higher-numbered chopstick.

Example:

P0 needs C0 and C4.
P0 picks C0 first, then C4.

P1 needs C0 and C1.
P1 picks C0 first, then C1.

P2 needs C1 and C2.
P2 picks C1 first, then C2.

This prevents circular wait.

3.14 Ordered Resource Algorithm

```
do {  
    think();  
  
    first = min(left_chopstick, right_chopstick);  
    second = max(left_chopstick, right_chopstick);  
  
    wait(first);  
    wait(second);  
  
    eat();  
  
    signal(second);  
    signal(first);  
  
} while (true);
```

3.15 Why Ordering Prevents Deadlock

Deadlock requires circular wait.

Circular wait means:

```
P0 waits for resource held by P1
P1 waits for resource held by P2
P2 waits for resource held by P3
...
Pn waits for resource held by P0
```

But if all philosophers acquire resources in increasing order, a cycle cannot form.

There cannot be a cycle where every process waits for a higher-numbered resource forever and eventually returns to a lower-numbered resource.

So circular wait is eliminated.

3.16 Solution 3: Odd-Even Strategy

Another simple method:

- Odd-numbered philosophers pick left chopstick first.
- Even-numbered philosophers pick right chopstick first.

Example:

```
if (philosopher_id % 2 == 0) {
    wait(right_chopstick);
    wait(left_chopstick);
} else {
    wait(left_chopstick);
    wait(right_chopstick);
}
```

This prevents all philosophers from picking the same side first.

3.17 Advantage of Odd-Even Strategy

Advantages:

- Simple.
- Breaks circular wait.
- Allows more concurrency than a global lock.

3.18 Disadvantage of Odd-Even Strategy

Disadvantages:

- Fairness still depends on scheduling.
- Starvation may still be possible if not implemented carefully.

- It is less general than strict resource ordering.

3.19 Solution 4: Monitor-Based Solution

A monitor is a high-level synchronization construct.

It combines:

- Mutual exclusion
- Condition variables

Each philosopher has a state:

```
enum { THINKING, HUNGRY, EATING } state[5];
```

Each philosopher also has a condition variable:

```
condition self[5];
```

3.20 Monitor Algorithm

```
monitor DiningPhilosophers {  
    enum { THINKING, HUNGRY, EATING } state[5];  
    condition self[5];  
  
    void pickup(int i) {  
        state[i] = HUNGRY;  
        test(i);  
  
        if (state[i] != EATING) {  
            self[i].wait();  
        }  
    }  
  
    void putdown(int i) {  
        state[i] = THINKING;  
  
        test((i + 4) % 5);  
        test((i + 1) % 5);  
    }  
  
    void test(int i) {  
        if (state[(i + 4) % 5] != EATING &&  
            state[i] == HUNGRY &&  
            state[(i + 1) % 5] != EATING) {  
  
            state[i] = EATING;  
            self[i].signal();  
        }  
    }  
}
```

```
    }  
  }  
}
```

Philosopher:

```
do {  
    think();  
    DiningPhilosophers.pickup(i);  
    eat();  
    DiningPhilosophers.putdown(i);  
} while (true);
```

3.21 Explanation of Monitor Solution

When a philosopher becomes hungry:

```
state[i] = HUNGRY;
```

Then the monitor checks whether both neighbors are not eating:

```
test(i);
```

If both neighbors are not eating, the philosopher can eat.

If a neighbor is eating, the philosopher waits:

```
self[i].wait();
```

When a philosopher finishes eating, they set their state back to thinking:

```
state[i] = THINKING;
```

Then they check whether their left or right neighbor can now eat:

```
test(left_neighbor);  
test(right_neighbor);
```

3.22 Why Neighbor Check Is Enough

A philosopher only shares chopsticks with immediate neighbors.

So philosopher i only conflicts with:

```
left neighbor  = (i + 4) % 5  
right neighbor = (i + 1) % 5
```

Non-neighboring philosophers can eat at the same time.

Example:

```
P0 and P2 can eat together.  
P1 and P3 can eat together.
```

3.23 Starvation in Dining Philosophers

Starvation occurs if a philosopher waits forever while others keep eating.

Even if a solution prevents deadlock, it may still allow starvation.

Example:

- P1 repeatedly eats.
- P3 repeatedly eats.
- P2 is always blocked because one of its neighbors is eating.

To prevent starvation, fairness mechanisms may be needed.

Examples:

- FIFO queue.
- Priority based on waiting time.
- Fair semaphores.
- Aging.

3.24 Deadlock vs Starvation

Deadlock:

```
Everyone is stuck.  
No process can proceed.
```

Starvation:

```
Some process waits forever.
```

Other processes may continue.

In Dining Philosophers:

- Deadlock means all philosophers wait forever.
- Starvation means one philosopher waits forever while others keep eating.

3.25 Important Interview Points

Why does the naive solution deadlock?

Because every philosopher can hold one chopstick and wait for another, creating circular wait.

Which deadlock condition should we break?

Usually circular wait is easiest to break.

Examples:

- Pick chopsticks in a global order.
- Use odd-even pickup order.
- Allow only 4 philosophers to compete.

Can a deadlock-free solution still cause starvation?

Yes.

Deadlock freedom does not automatically mean starvation freedom.

Why not use one global mutex?

You can, but it is inefficient.

Example:

```
wait(global_mutex);
pick_chopsticks();
eat();
put_chopsticks();
signal(global_mutex);
```

This prevents deadlock, but only one philosopher can eat at a time.

It destroys concurrency because non-neighboring philosophers could have eaten together.

What is the best solution?

It depends on the goal:

- For simplicity: room semaphore solution.

- For deadlock prevention: resource ordering.
- For structured synchronization: monitor solution.
- For fairness: queue-based monitor or fair lock.

3.26 Common Mistakes

Common errors:

- Thinking deadlock and starvation are the same.
- Preventing deadlock but ignoring starvation.
- Allowing neighboring philosophers to eat together.
- Using a global lock unnecessarily.
- Forgetting that each philosopher needs two resources.
- Forgetting circular wait is the key issue.

3.27 Summary

The Dining Philosophers Problem teaches:

- Deadlock conditions.
- Resource allocation problems.
- Circular wait.
- Starvation.
- Fairness.
- Why resource acquisition order matters.

4. Quick Comparison Table

Problem	Main Shared Resource	Main Risk	Common Solution
Producer-Consumer	Bounded buffer	Overflow, underflow, race condition	empty, full, mutex semaphores
Readers-Writers	Shared data/file/database	Inconsistent reads, writer starvation	Read-write locks, semaphores
Dining Philosophers	Chopsticks/resources	Deadlock, starvation	Ordering, room semaphore, monitor

5. Key Terms

Race Condition

A race condition occurs when the result of a program depends on the timing of concurrent operations.

Critical Section

A critical section is the part of a program where shared data is accessed.

Mutual Exclusion

Mutual exclusion ensures that only one process enters a critical section at a time.

Semaphore

A semaphore is a synchronization variable used to control access to shared resources.

Two common operations:

```
wait(S); // also called P, down, acquire
signal(S); // also called V, up, release
```

Binary Semaphore

A binary semaphore has values 0 or 1.

It is often used like a mutex.

Counting Semaphore

A counting semaphore can have values greater than 1.

It is used to count available resources.

Example:

```
empty = number of empty slots
```

Mutex

A mutex is a lock used to provide mutual exclusion.

Only the thread that locks a mutex should unlock it.

Deadlock

Deadlock occurs when a set of processes wait forever because each process is waiting for a resource held by another process in the set.

Starvation

Starvation occurs when a process waits indefinitely because other processes keep getting access before it.

Bounded Waiting

Bounded waiting means there is a limit on how long a process has to wait before entering its critical section.

6. Common Interview Questions

6.1 Producer-Consumer

Q1. What is the Producer-Consumer Problem?

It is a synchronization problem where producers generate items and place them in a shared buffer, while consumers remove items from that buffer. The challenge is to prevent race conditions, buffer overflow, and buffer underflow.

Q2. Which semaphores are used?

Usually:

```
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```

Q3. Why is `empty` initialized to `n`?

Because initially all `n` buffer slots are empty.

Q4. Why is `full` initialized to `0`?

Because initially there are no items in the buffer.

Q5. What happens if the producer does not wait on `empty`?

The producer may insert into a full buffer, causing overflow or data loss.

Q6. What happens if the consumer does not wait on `full`?

The consumer may remove from an empty buffer, causing underflow or invalid reads.

6.2 Readers-Writers

Q1. What is the Readers-Writers Problem?

It is a synchronization problem where multiple readers can read shared data at the same time, but writers need exclusive access.

Q2. Why can multiple readers read simultaneously?

Because reading does not change the shared data.

Q3. Why does a writer need exclusive access?

Because writing modifies data and can cause inconsistency if mixed with other reads or writes.

Q4. What is reader priority?

Reader priority means readers are allowed to enter as long as no writer is currently writing. This can starve writers.

Q5. What is writer priority?

Writer priority means once a writer is waiting, new readers are blocked. This can starve readers.

Q6. How can starvation be avoided?

By using a fair scheduling policy, FIFO queue, fair read-write lock, or turnstile semaphore.

6.3 Dining Philosophers

Q1. What is the Dining Philosophers Problem?

It is a synchronization problem where philosophers need two shared chopsticks to eat. The challenge is to avoid deadlock and starvation.

Q2. Why does the naive solution cause deadlock?

If all philosophers pick up one chopstick and wait for the other, a circular wait forms and nobody can continue.

Q3. What are the four deadlock conditions?

The four necessary deadlock conditions are:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Q4. How can deadlock be prevented?

Common ways:

- Allow only four philosophers to try eating.
- Use resource ordering.
- Use odd-even pickup order.
- Use a monitor.

Q5. Can starvation occur even if deadlock is prevented?

Yes. A philosopher may wait forever if neighbors keep getting chances to eat.

7. Final Exam/Placement Revision Points

Remember these points:

- Producer-Consumer focuses on buffer synchronization.
- Readers-Writers focuses on shared reading and exclusive writing.
- Dining Philosophers focuses on deadlock and resource allocation.
- Mutex protects critical sections.
- Counting semaphores count available resources.
- Deadlock and starvation are different.
- Avoiding deadlock does not always avoid starvation.
- Fairness often requires extra mechanisms.
- Lock ordering is a common technique to prevent circular wait.
- In interviews, always explain why each semaphore or lock is needed.